

INFORME OWASP ASVS 4.0.3

IMPLEMENTACIÓN DE DIEZ CONTROLES DE SEGURIDAD (UNO POR CAPÍTULO)

Proyecto: Sistema de Control y Monitoreo Línea 61	Estándar: OWASP ASVS v4.0.3 (Nivel 1 y 2)
Framework: Laravel 11 / PHP 8.3 / Sanctum / Spatie Roles	Estado de Auditoría: 100% Implementado y Verificado

Resumen Ejecutivo de los 10 Controles Seleccionados

Nº	Capítulo ASVS 4.0.3	Control	Objetivo / Mitigación Implementada	Estado
1	Cap. 2: V2 Autenticación	V2.1.1/V2.1.7	Contraseña obligatoria con mínimo 12 caracteres	Implementado
2	Cap. 3: V3 Gestión Sesiones	V3.2.1	Regeneración de sesión al login y logout seguro	Implementado
3	Cap. 4: V4 Control Acceso	V4.1.1/V4.3.1	RBAC y mínimo privilegio (rutas admin y conductor)	Implementado
4	Cap. 5: V5 Validación y Sanit.	V5.1.1	Validación de entradas en servidor	Implementado
5	Cap. 6: V6 Criptografía	V6.2.1	Almacenamiento seguro de passwords con Hash Bcrypt	Implementado
6	Cap. 7: V7 Manejo Errores	V7.1.1/V7.1.2	Vistas de error limpias sin fugar stack trace / rutas	Implementado
7	Cap. 8: V8 Protección Datos	V8.2.1	Ocultamiento de passwords y tokens en JSON / API	Implementado
8	Cap. 9: V9 Comunicaciones	V9.1.2	Cookies de transporte con flags HttpOnly y SameSite	Implementado
9	Cap. 11: V11 Lógica Negocio	V11.1.4	Rate Limiting anti-fuerza bruta en login (5 intentos)	Implementado
10	Cap. 13: V13 APIs y Servicios	V13.1.4	Protección de endpoints REST con Tokens Sanctum	Implementado

CAPÍTULO 2: V2 AUTENTICACIÓN (AUTHENTICATION)

Control V2.1.1 / V2.1.7: Requisito de Contraseña de Mínimo 12 Caracteres

Descripción y Justificación de Seguridad (ASVS 4.0.3):

OWASP ASVS 4.0.3 establece en V2.1.1 y V2.1.7 que las contraseñas deben tener una longitud mínima suficiente para resistir ataques de diccionario y fuerza bruta fuera de línea (mínimo 12 caracteres para aplicaciones empresariales). Se prohíben claves cortas o triviales.

Archivos, Funciones y Líneas Modificadas:

- **app/Providers/AppServiceProvider.php** (líneas 19-25): Definición global de Password::defaults(fn() => Password::min(12)).
- **app/Http/Controllers/Auth/RegisteredUserController.php** (línea 36): Validación en el registro de nuevos usuarios.
- **app/Http/Controllers/Auth/PasswordController.php** (línea 20): Validación en la actualización de contraseñas de usuarios existentes.

Código Implementado:

```
// app/Providers/AppServiceProvider.php
public function boot(): void {
    \Illuminate\Validation\Rules\Password::defaults(function () {
        return \Illuminate\Validation\Rules\Password::min(12); // Exige mínimo 12 caracteres (ASVS V2.1.1)
    });
}
```

¿Cómo se prueba en la defensa en vivo? Ir al formulario de registro (/register) o perfil de usuario (/profile). Intentar registrar o actualizar una contraseña corta de 8 caracteres (ej. admin123). El servidor rechaza la petición mostrando el mensaje de validación: 'The password field must be at least 12 characters'. Al colocar 12 o más caracteres, se procesa con éxito.

CAPÍTULO 3: V3 GESTIÓN DE SESIONES (SESSION MANAGEMENT)

Control V3.2.1: Regeneración de Identificador de Sesión (Anti-Fijación de Sesión)

Descripción y Justificación de Seguridad (ASVS 4.0.3):

ASVS V3.2.1 exige que el identificador de sesión sea regenerado inmediatamente tras una autenticación exitosa para prevenir ataques de Fijación de Sesión (Session Fixation), y que sea destruido en el cierre de sesión (logout) evitando la reutilización de identificadores.

Archivos, Funciones y Líneas Modificadas:

- **app/Http/Controllers/Auth/AuthenticatedSessionController.php** (líneas 29 y 41-43): Invocación de \$request->session()->regenerate() en login y invalidate() en logout.
- **config/session.php** (líneas 30-35): Tiempo de vida de sesión controlado y expiración de cookies.

Código Implementado:

```
// app/Http/Controllers/Auth/AuthenticatedSessionController.php
public function store(LoginRequest $request): RedirectResponse {
    $request->authenticate();
    $request->session()->regenerate(); // ASVS V3.2.1: Regenera token de sesión
    return redirect()->intended(route('dashboard'));
}
```

¿Cómo se prueba en la defensa en vivo? Abrir el navegador, presionar **F12** e ir a *Application > Storage > Cookies*. Copiar el valor de la cookie laravel_session antes de iniciar sesión. Iniciar sesión: se verifica inmediatamente que el valor de la cookie cambió por completo. Al presionar 'Cerrar Sesión', la sesión anterior queda invalidada en el servidor.

CAPÍTULO 4: V4 CONTROL DE ACCESO (ACCESS CONTROL)

Control V4.1.1 / V4.3.1: Mínimo Privilegio y Separación de Roles (RBAC)

Descripción y Justificación de Seguridad (ASVS 4.0.3):

ASVS V4.1.1 y V4.3.1 exigen aplicar el principio de menor privilegio, separando estrictamente funciones de administración. En este proyecto: (1) Un conductor NO puede asignarse turnos a sí mismo; (2) Usuarios y roles son exclusivos de Admin; (3) Propietarios solo acceden a monitoreo de recorrido.

Archivos, Funciones y Líneas Modificadas:

- **routes/web.php** (líneas 29-54): Rutas agrupadas con role:admin|fiscalizador, role:admin y role:admin|fiscalizador|dueño.
- **routes/api.php** (líneas 22-31): Endpoints administrativos bajo role:admin.
- **tests/Feature/Api/ConductorCrudTest.php** (líneas 58-65): Pruebas automáticas de rechazo HTTP 403.

Código Implementado:

```
// routes/web.php
Route::middleware('role:admin|fiscalizador')->group(function () {
    Route::resource('asignacion-turno', AsignacionTurnoController::class); // Conductor NO puede autoasignarse
});
Route::middleware('role:admin')->group(function () {
    Route::resource('users', UserController::class);
    Route::resource('roles', RolesController::class);
});
```

¿Cómo se prueba en la defensa en vivo? Iniciar sesión con una cuenta de rol *conductor*. Intentar acceder escribiendo directamente en la barra de URL `http://127.0.0.1:8000/users` o `/asignacion-turno`. El sistema bloquea el acceso mostrando la pantalla oficial **HTTP 403 Forbidden**. Al entrar como *admin*, el acceso es autorizado.

CAPÍTULO 5: V5 VALIDACIÓN, SANITIZACIÓN Y CODIFICACIÓN (VALIDATION & ENCODING)

Control V5.1.1: Validación de Entradas del Lado Servidor

Descripción y Justificación de Seguridad (ASVS 4.0.3):

ASVS V5.1.1 exige validar todos los datos recibidos en el servidor antes de procesarlos o guardarlos. En el módulo de paradas, Leaflet y Nominatim ayudan a seleccionar la ubicación y completar latitud/longitud, pero Laravel vuelve a validar esos datos antes de guardar. Así se rechazan paradas sin ubicación, coordenadas fuera del rango geográfico válido y estados no permitidos.

Archivos, Funciones y Líneas Modificadas:

- **app/Http/Controllers/ParadaController.php** (store): Reglas para validar nombre, referencia, latitud, longitud y estado antes de crear.
- **app/Http/Controllers/ParadaController.php** (update): Aplica las mismas reglas antes de modificar una parada existente.
- **resources/views/parada/form.blade.php**: Usa Leaflet y Nominatim para seleccionar la ubicación y rellenar los campos ocultos de latitud/longitud.

Código Implementado:

```
// app/Http/Controllers/ParadaController.php
$request->validate([
    'nombre' => ['required', 'string', 'max:100'],
    'referencia' => 'nullable|string|max:255',
    'latitud' => 'required|numeric|between:-90,90',
    'longitud' => 'required|numeric|between:-180,180',
    'estado' => 'required|in:activo,inactivo',
]);
```

¿Cómo se prueba en la defensa en vivo? Iniciar sesión como administrador, ir a Paradas, escribir un nombre y presionar guardar sin seleccionar ningún punto en el mapa. El sistema rechaza el formulario porque latitud y longitud son obligatorias. Luego seleccionar una ubicación en el mapa: JavaScript completa los campos con geocodificación inversa y el registro se guarda. Como prueba adicional, desde DevTools se puede cambiar latitud a 120 o longitud a 200; Laravel lo rechaza por estar fuera del rango permitido.

CAPÍTULO 6: V6 CRIPTOGRAFÍA ALMACENADA (STORED CRYPTOGRAPHY)**Control V6.2.1: Almacenamiento Seguro de Contraseñas (Hash Bcrypt)****Descripción y Justificación de Seguridad (ASVS 4.0.3):**

ASVS V6.2.1 exige que todas las contraseñas se almacenen utilizando una función hash criptográfica unidireccional con sal (*salt*) automático resistente a ataques por GPU (Bcrypt con factor de costo 12). Se prohíbe terminantemente el almacenamiento en texto plano o con algoritmos obsoletos como MD5 o SHA1.

Archivos, Funciones y Líneas Modificadas:

- **app/Models/User.php** (líneas 27-28): Cast nativo 'password' => 'hashed'.
- **config/hashing.php**: Driver por defecto configurado en bcrypt con costo 12.

Código Implementado:

```
// app/Models/User.php
protected function casts(): array {
    return [
        'password' => 'hashed', // Aplica Bcrypt unidireccional con salt automático
    ];
}
```

¿Cómo se prueba en la defensa en vivo? Abrir la base de datos (phpMyAdmin o terminal con php artisan tinker) y consultar la contraseña del usuario. Se demuestra que el valor almacenado es un hash del tipo \$2y\$12\$... imposible de revertir a texto plano.

CAPÍTULO 7: V7 MANEJO DE ERRORES Y REGISTRO (ERROR HANDLING & LOGGING)**Control V7.1.1 / V7.1.2: Respuestas de Error Limpias sin Fuga de Información****Descripción y Justificación de Seguridad (ASVS 4.0.3):**

ASVS V7.1.1 y V7.1.2 prohíben que los mensajes de error revelen detalles de implementación interna (rutas absolutas de servidor, versiones de librerías, credenciales de BD o volcados de excepciones). La aplicación debe responder con páginas de error personalizadas y genéricas.

Archivos, Funciones y Líneas Modificadas:

- **resources/views/errors/403.blade.php**: Vista amigable para accesos denegados.
- **resources/views/errors/404.blade.php**: Vista para recursos inexistentes.
- **resources/views/errors/500.blade.php**: Vista controlada para excepciones de servidor.

Código Implementado:

```
// resources/views/errors/403.blade.php
@extends('errors::minimal')
@section('title', __('Forbidden'))
@section('code', '403')
@section('message', 'Acceso no autorizado. El recurso solicitado.')

```

¿Cómo se prueba en la defensa en vivo? Escribir en la URL cualquier dirección que no exista (ej. http://127.0.0.1:8000/ruta-inexistente-xyz). El sistema responde con una interfaz limpia de error 404 sin exponer trazas de código ni rutas del sistema de archivos de Windows/Laragon.

CAPÍTULO 8: V8 PROTECCIÓN DE DATOS (DATA PROTECTION)

Control V8.2.1: Ocultamiento de Secretos en Serializaciones JSON

Descripción y Justificación de Seguridad (ASVS 4.0.3):

ASVS V8.2.1 exige que los datos confidenciales (tales como contraseñas hasheadas, tokens de recuperación o llaves de sesión) nunca sean expuestos en las respuestas de la API ni filtrados en respuestas JSON enviadas al frontend.

Archivos, Funciones y Líneas Modificadas:

- **app/Models/User.php** (línea 17): Atributo Eloquent #[Hidden(['password', 'remember_token'])].
- **app/Http/Controllers/Api/AuthController.php** (línea 29): Método me() que serializa el usuario seguro.

Código Implementado:

```
// app/Models/User.php
#[Hidden(['password', 'remember_token'])] // Protege contra exposición en respuestas JSON
class User extends Authenticatable implements MustVerifyEmail { ... }
```

¿Cómo se prueba en la defensa en vivo? Realizar una petición a GET /api/me con el token de autenticación (o ver la respuesta de POST /api/login). El payload JSON devuelto muestra name, email, roles, pero el atributo password está completamente ausente.

CAPÍTULO 9: V9 COMUNICACIONES (COMMUNICATIONS)

Control V9.1.2: Banderas de Seguridad HttpOnly y SameSite en Cookies

Descripción y Justificación de Seguridad (ASVS 4.0.3):

ASVS V9.1.2 exige que las cookies de sesión se configuren con la bandera HttpOnly para impedir que scripts del cliente lean la cookie mediante document.cookie, y la directiva SameSite=Lax para proteger las peticiones contra falsificación de petición en sitios cruzados (CSRF).

Archivos, Funciones y Líneas Modificadas:

- **config/session.php** (líneas 172, 185 y 202): http_only => true, same_site => 'lax', secure => env(...).

Código Implementado:

```
// config/session.php
'http_only' => env('SESSION_HTTP_ONLY', true), // Bloquea acceso desde JavaScript
'same_site' => env('SESSION_SAME_SITE', 'lax'), // Mitiga CSRF en peticiones cruzadas
'secure' => env('SESSION_SECURE_COOKIE', env('APP_ENV') === 'production'),
```

¿Cómo se prueba en la defensa en vivo? Abrir la consola de desarrollador en el navegador (F12 > Console) con la sesión iniciada. Escribir document.cookie y presionar Enter. La cookie de sesión laravel_session no aparece, demostrando que está protegida por HttpOnly.

CAPÍTULO 11: V11 LÓGICA DE NEGOCIO (BUSINESS LOGIC)**Control V11.1.4: Límite de Intentos y Anti-Fuerza Bruta en Login****Descripción y Justificación de Seguridad (ASVS 4.0.3):**

ASVS V11.1.4 exige defensas automatizadas contra abuso repetitivo de la lógica de negocio. En la autenticación, se debe limitar la tasa de intentos (*Rate Limiting*) bloqueando temporalmente a usuarios o atacantes que intenten adivinar contraseñas de forma consecutiva.

Archivos, Funciones y Líneas Modificadas:

- **app/Http/Requests/Auth/LoginRequest.php** (líneas 62-78): Método `ensureIsNotRateLimited()` con máximo 5 intentos por correo/IP.
- **routes/api.php** (línea 15): Endpoint de login protegido con middleware `throttle:5,1`.

Código Implementado:

```
// app/Http/Requests/Auth/LoginRequest.php
public function ensureIsNotRateLimited(): void {
    if (!RateLimiter::tooManyAttempts($this->throttleKey(), 5)) return;
    event(new Lockout($this));
    $seconds = RateLimiter::availableIn($this->throttleKey());
    throw ValidationException::withMessages(['email' => trans('auth.throttle', ['seconds' => $seconds])]);
}
```

¿Cómo se prueba en la defensa en vivo? Ir al formulario de inicio de sesión e ingresar 5 veces seguidas una contraseña incorrecta. Al 6to intento, el sistema bloquea inmediatamente la cuenta mostrando: *'Demasiados intentos de acceso. Por favor inténtelo de nuevo en 60 segundos'* (HTTP 429 Too Many Requests).

CAPÍTULO 13: V13 APIS Y SERVICIOS WEB (API & WEB SERVICE)**Control V13.1.4: Protección de APIs con Tokens Bearer (Laravel Sanctum)****Descripción y Justificación de Seguridad (ASVS 4.0.3):**

ASVS V13.1.4 exige que todos los servicios web y endpoints REST que expongan información de negocio o datos operativos requieran autenticación explícita y no puedan ser accedidos de forma anónima sin un token criptográfico validado.

Archivos, Funciones y Líneas Modificadas:

- **routes/api.php** (líneas 18-45): Rutas protegidas bajo el middleware `auth:sanctum`.
- **app/Http/Controllers/Api/AuthController.php** (líneas 18-24): Emisión y revocación de Bearer tokens.

Código Implementado:

```
// routes/api.php
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::get('/mis/asignaciones', [AsignacionTurnoApi::class, 'misAsignaciones']);
    // Endpoints GPS de conductores protegidos
});
```

¿Cómo se prueba en la defensa en vivo? Enviar una petición GET `http://127.0.0.1:8000/api/mis/asignaciones` desde Postman o navegador sin enviar token. El servidor responde inmediatamente con **HTTP 401 Unauthorized** (`{'message': 'Unauthenticated'}`). Al enviar el token Bearer emitido en el login, responde con **HTTP 200 OK** y los datos correspondientes.